

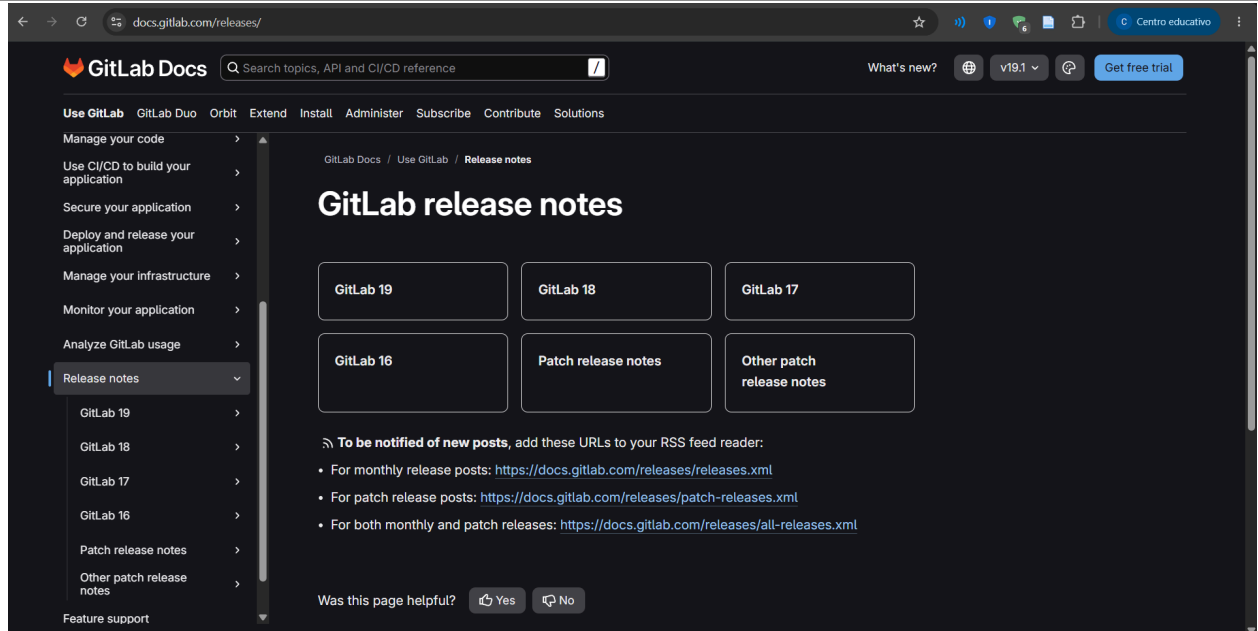
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Mantenimiento, Configuración, Despliegue y Evolución de Software				
TÍTULO DE LA PRÁCTICA:	<i>El estándar IEEE 1219</i>				
NÚMERO DE PRÁCTICA:	<i>07</i>	AÑO LECTIVO:	<i>2026-A</i>	NRO. SEMESTRE:	<i>IX (novenio)</i>
FECHA DE PRESENTACIÓN		HORA DE PRESENTACIÓN	<i>23:59</i>		
INTEGRANTE (s): Salhua Apfata Cristian Roberto				NOTA:	
DOCENTE(s): <i>M. Sc. Ing. Giovanni Cabrera Málaga</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS <i>Tomando como base el archivo entregado se pide realizar el presente Laboratorio.</i>
II. SOLUCIÓN DEL CUESTIONARIO <i>Seleccione un sistema de software real (puede ser un sistema legado o una aplicación conocida).</i> El sistema de software real seleccionado es GitLab, una plataforma ampliamente utilizada para gestión del ciclo de vida del desarrollo de software, repositorios Git, integración continua, despliegue, seguimiento de incidencias y administración de releases. La elección es pertinente porque se trata de un sistema real, conocido, con documentación técnica pública y con políticas explícitas sobre mantenimiento y liberación de versiones. GitLab ofrece una base adecuada para esta práctica porque su documentación describe de manera formal cómo maneja releases mensuales, patch releases e internal releases. Además, especifica que sus versiones siguen semantic versioning, indicando que las versiones MAJOR pueden introducir cambios incompatibles, las MINOR incorporan nuevas funcionalidades compatibles y las PATCH contienen únicamente correcciones de errores. Esto lo convierte en un ejemplo especialmente útil para estudiar mantenimiento correctivo, preventivo y procesos de liberación controlada.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2



Documentación oficial donde se observan los tipos de release y versionado de GitLab.

■ *Identifique problemas o necesidades de mantenimiento en ese sistema.*

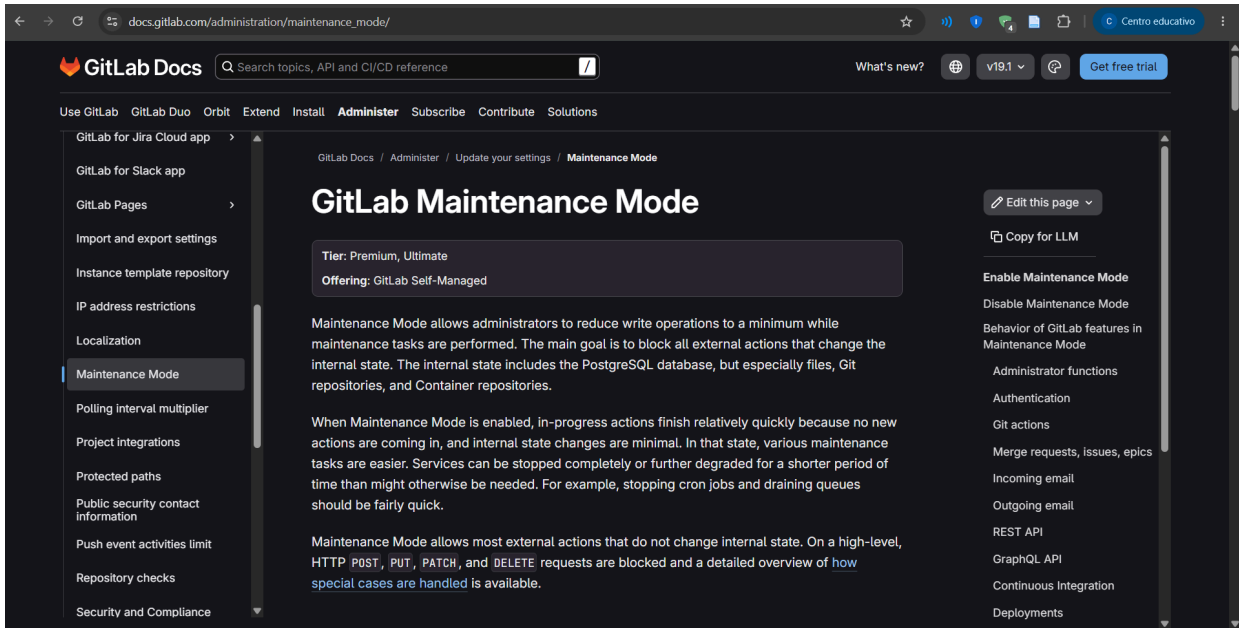
A partir de la documentación oficial, puede observarse que GitLab requiere mantenimiento continuo por la naturaleza de sus releases y por su compromiso con versiones mantenidas. La existencia de patch releases para correcciones de errores y actualizaciones de seguridad demuestra que el sistema necesita atención permanente para conservar estabilidad, seguridad y confianza del usuario.

Una primera necesidad es el mantenimiento correctivo, evidente cuando GitLab publica patch releases para resolver regresiones, errores o vulnerabilidades. Por ejemplo, la documentación de releases indica que las versiones patch contienen bug fixes y security fixes, y algunas publicaciones de release señalan actualizaciones urgentes recomendadas para instalaciones self-managed.

Una segunda necesidad es el mantenimiento preventivo, ya que GitLab define políticas de mantenimiento, versiones mantenidas y backports para evitar que problemas conocidos se propaguen o permanezcan sin corrección. La política oficial distingue entre versiones actualmente mantenidas y versiones anteriores que reciben únicamente ciertos tipos de corrección, especialmente de seguridad.

También puede identificarse una necesidad de mantenimiento perfectivo, porque las releases mensuales incorporan nuevas funcionalidades, mejoras de rendimiento y ajustes que refinan el comportamiento del sistema sin cambiar necesariamente su propósito principal. Finalmente, existe un componente adaptativo, dado que GitLab debe evolucionar para responder a nuevas exigencias operativas, prácticas de despliegue, seguridad y automatización del desarrollo, como se aprecia en su soporte a releases por CI/CD, asset links, milestones y mecanismos de control como deploy freezes.

En términos técnicos, estas necesidades se parecen a las que enfrentan sistemas empresariales contruidos con stacks modernos como .NET, Spring Boot o Node.js: alta frecuencia de cambios, exigencia de trazabilidad, necesidad de evitar regresiones y presión por desplegar de forma segura sin afectar módulos ya estables.



■ Relacionar las fases del estándar IEEE 1219 con un posible plan de mantenimiento para ese sistema.

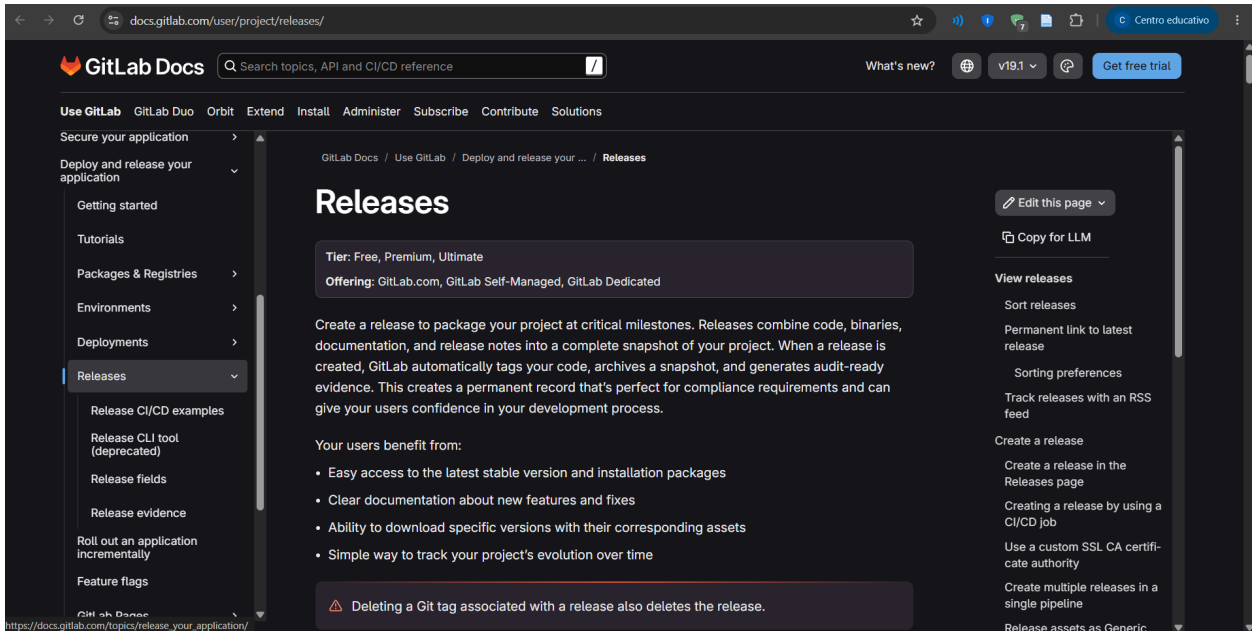
A continuación, se relacionan las fases del estándar IEEE 1219 con un posible plan de mantenimiento para GitLab. El objetivo no es reproducir el proceso interno completo de la empresa desarrolladora, sino mostrar cómo podría modelarse un mantenimiento ordenado tomando como base la información pública disponible sobre releases, versiones mantenidas y evidencia de liberación.

Fase IEEE 1219	Aplicación en GitLab	Resultado esperado
Identificación y clasificación	Registrar errores, vulnerabilidades o regresiones reportadas; clasificar si se trata de corrección, mejora o cambio de seguridad.	Lista de incidencias priorizadas.
Análisis	Evaluar el impacto del defecto, la versión afectada y la posibilidad de aplicar backport o corrección directa.	Informe de impacto y viabilidad.
Diseño	Definir el cambio, el alcance técnico, la estrategia de pruebas y la forma de liberación.	Diseño del mantenimiento y plan de validación.

Implementación	Aplicar la corrección en la rama estable correspondiente, registrar el cambio y preparar el parche.	Cambio implementado y trazado.
Pruebas del sistema	Verificar que la corrección funcione y no introduzca regresiones en funciones críticas.	Resultados de pruebas y validación técnica.
Pruebas de aceptación	Confirmar que el cambio satisface el objetivo funcional o de seguridad previsto.	Aprobación del mantenimiento.
Liberación del producto	Publicar release con tag, notas, activos y evidencia auditable.	Nueva versión disponible y documentada.

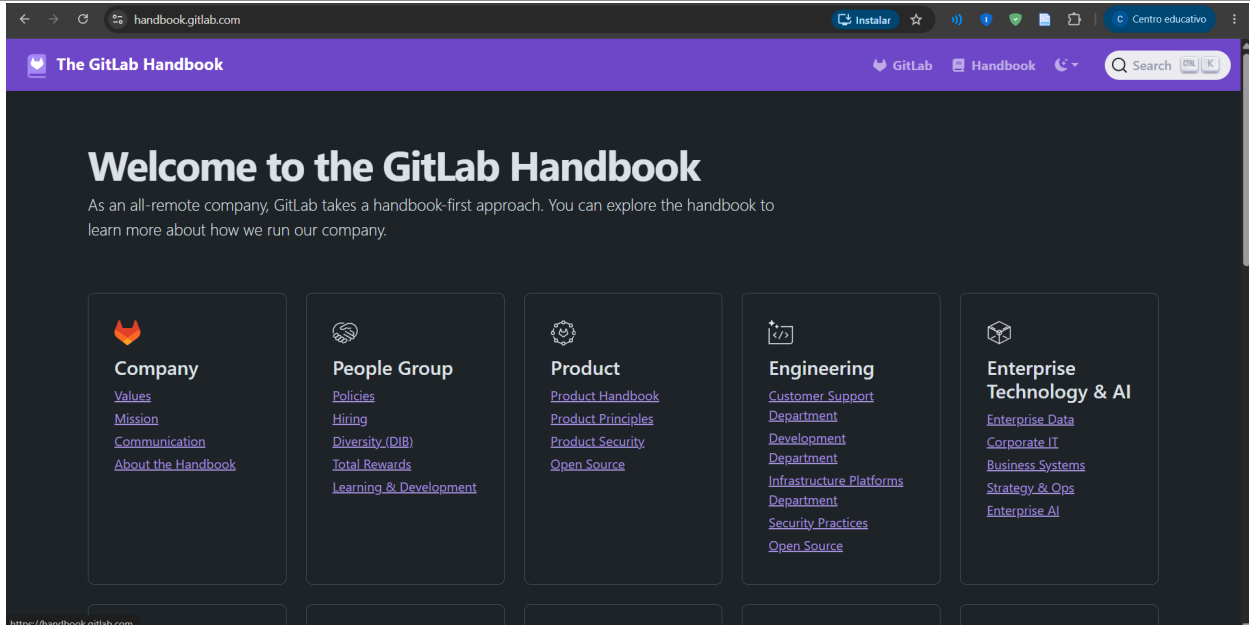
Esta relación es coherente con la documentación oficial de GitLab, donde se observa que las releases combinan código, binarios, documentación y release notes, y además generan evidencia de release útil para procesos de auditoría. Asimismo, la política de patch releases y backports refuerza la idea de que el mantenimiento no se limita a “corregir código”, sino que incluye clasificación, validación, documentación y control de configuración.

■ Colocar evidencias.

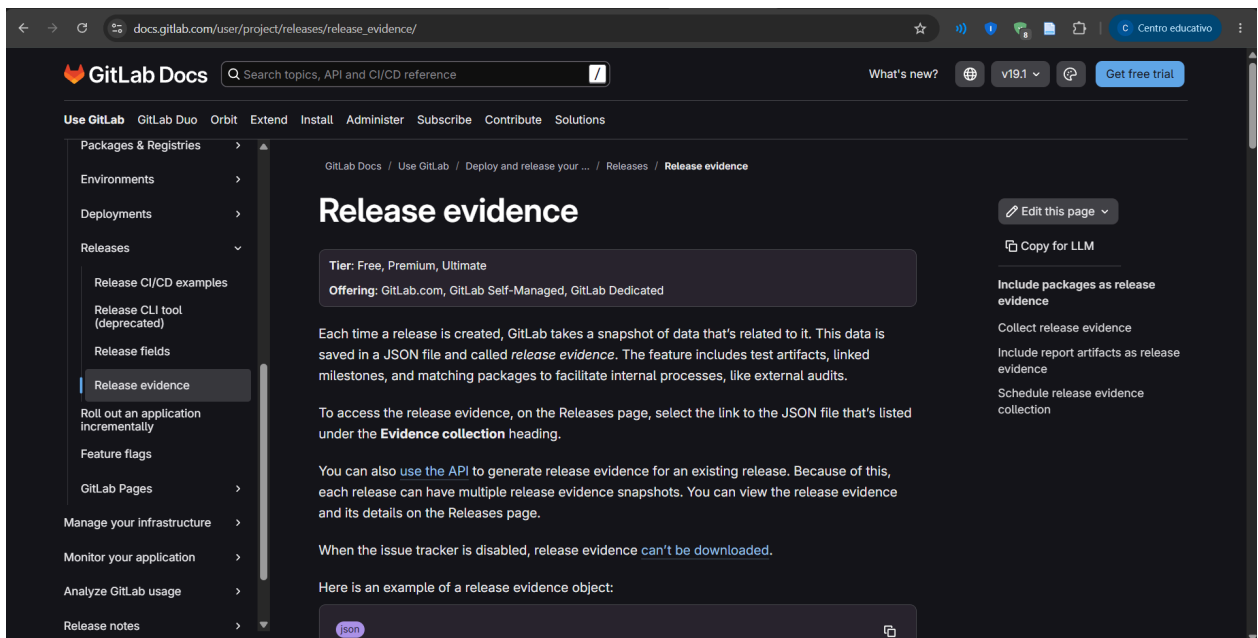


GitLab Releases, donde se explica que las releases agrupan código, binarios, documentación y notas de versión .

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

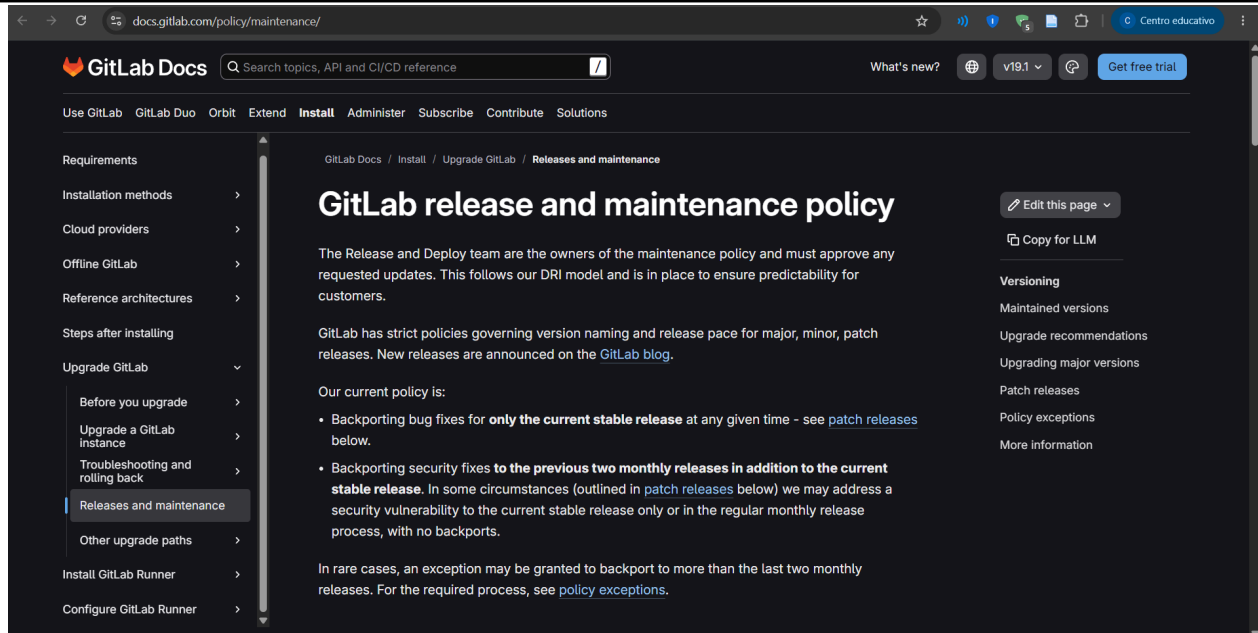


GitLab Handbook o política de releases, donde se describen las monthly releases, patch releases e internal releases.



Documentación de release evidence, donde se explica el uso de evidencia en JSON para auditoría.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6



Política de mantenimiento, donde se identifican versiones mantenidas y backports.

• **Caso de Estudio:**

La empresa Finanzas Globales S.A. utiliza desde hace 7 años un sistema llamado FINCONTROL, desarrollado en .NET, para gestionar préstamos, pagos y reportes financieros. El sistema es crítico, se utiliza por más de 50 agencias en tiempo real y está conectado a la base central del banco. Recientemente, se identificó un error en el módulo de generación de reportes fiscales. Este error genera reportes duplicados en ciertas condiciones y afecta la información enviada a los entes reguladores. El equipo de TI debe aplicar una corrección urgente, sin afectar el resto del sistema, especialmente el módulo de pagos, que es muy sensible.

Condiciones actuales:

- *El sistema no tiene pruebas automatizadas.*
- *El código fue modificado varias veces sin un control de versiones adecuado.*
- *El equipo de mantenimiento actual no participó en el desarrollo original.*
- *El módulo de reportes comparte funciones con otros módulos críticos.*

La empresa Finanzas Globales S.A. utiliza desde hace siete años el sistema FINCONTROL, desarrollado en .NET, para gestionar préstamos, pagos y reportes financieros. El error detectado en el módulo de reportes fiscales, que genera duplicidad bajo ciertas condiciones, constituye un caso claro de mantenimiento correctivo, pero con alta criticidad porque el sistema es usado por más de 50 agencias y está conectado a la base central del banco.

El escenario se vuelve más delicado debido a cuatro factores: no existen pruebas automatizadas, el código carece de trazabilidad sólida por mal control de versiones, el equipo actual no desarrolló el sistema originalmente y el módulo de reportes comparte funciones con componentes críticos. Desde la

perspectiva del mantenimiento, esto obliga a trabajar con una estrategia de mínima invasión, fuerte análisis de impacto y documentación estricta del cambio.

Fase IEEE 1219	Aplicación en FINCONTROL	Entregable esperado
Identificación y clasificación	Registrar la duplicación de reportes fiscales y clasificarla como incidencia correctiva crítica.	Solicitud de mantenimiento priorizada.
Análisis	Revisar condiciones de duplicación, dependencias compartidas y riesgo sobre pagos.	Informe de análisis técnico.
Diseño	Definir parche puntual, criterios de prueba, aislamiento del cambio y rollback.	Diseño de mantenimiento.
Implementación	Corregir el componente afectado en entorno controlado y registrar versión interna.	Parche implementado.
Pruebas del sistema	Ejecutar pruebas funcionales y de regresión sobre reportes y pagos.	Evidencia de pruebas.
Pruebas de aceptación	Validación por responsables financieros o regulatorios.	Conformidad del usuario clave.
Liberación	Desplegar parche, documentar cambio y resguardar respaldo.	Release interna del mantenimiento.

En un entorno .NET empresarial, sería razonable revisar clases compartidas, servicios internos, consultas a base de datos y rutinas reutilizadas entre reportes y pagos. Si el defecto se origina en una función compartida, una corrección apresurada puede resolver la duplicación fiscal pero introducir fallas financieras más graves. Por ello, la lógica de IEEE 1219 encaja bien en este caso: primero entender y aislar, luego corregir y validar.

Seleccione 2 formularios de la lista para poder recabar información necesaria donde se realiza un proyecto de mantenimiento para el software FINCONTROL:

- *Log de pruebas – IEEE 829-1983*
- *Informe de incidencias de pruebas – IEEE 829-1983*
- *Informe de resumen de pruebas – IEEE 829-1983*

- *Especificación del diseño de pruebas – IEEE 829-1983*
- *Solicitud de Cambio del Software/Sistema – IEEE 1042-1987*
- *Autorización de Cambio del Software – IEEE 1042-1987.*

Los dos formularios más adecuados para el proyecto de mantenimiento de FINCONTROL son los siguientes:

Formulario	Estándar	Justificación
Solicitud de Cambio del Software/Sistema	IEEE 1042-1987	Formaliza el requerimiento de mantenimiento, documenta impacto, prioridad, riesgo y módulos afectados.
Informe de incidencias de pruebas	IEEE 829-1983	Registra defectos detectados durante la validación del parche y permite decidir si el cambio puede liberarse.

La Solicitud de Cambio del Software/Sistema debe utilizarse porque el problema no puede tratarse como una intervención informal. En una aplicación financiera, un error en reportes fiscales afecta cumplimiento regulatorio, credibilidad institucional y consistencia de datos. Por ello, el cambio debe quedar identificado, priorizado, autorizado y relacionado con los componentes impactados.

El Informe de incidencias de pruebas también es fundamental, porque FINCONTROL no cuenta con pruebas automatizadas y el cambio podría generar efectos colaterales en módulos compartidos. Este formulario permite registrar de forma objetiva qué fallos aparecen durante la validación, en qué condiciones se reproducen y si el sistema está listo para producción o requiere nuevas iteraciones.

Estos dos formularios son los más apropiados porque cubren dos momentos críticos del mantenimiento: la decisión formal de cambiar y la validación real del resultado. En un sistema bancario, donde una corrección apresurada puede comprometer pagos o reportes regulatorios, trabajar sin estos documentos sería técnicamente riesgoso y administrativamente débil.

EJEMPLO DE FORMULARIO DE SOLICITUD DE CAMBIOS

FORMULARIO DE SOLICITUD DE CAMBIO

DETALLES DEL CAMBIO	Nombre del proyecto	Más cerca del sol		Cambio N.º	1011
	Nombre del cambio	Versión del producto de SP 3.2		Fecha de solicitud	11/11/20XX
	Solicitado por	Lori Garcia		Fecha en que se necesita	01/01/20XX
	Información de contacto de quien solicita		lgarcia@email.com		
PRIORIDAD		☒ ALTA	☐ MEDIA	☐ BAJA	
Descripción del cambio		Queremos aumentar la clasificación de la eficiencia de nuestros paneles solares residenciales del 22 al 25 por ciento.			
Motivo del cambio		La investigación de mercado demuestra que cada vez más clientes residenciales demandan paneles solares más eficientes. Al incrementar nuestra clasificación de la eficiencia, seremos más competitivos y demostraremos que somos una empresa innovadora.			
IMPACTOS DEL CAMBIO	Alcance	El alcance no variará.			
	Entregas	Deberemos actualizar el diseño y las especificaciones de nuestro producto.			
	Costo	Necesitaremos incrementar el presupuesto para la investigación, el desarrollo y las pruebas adicionales.			
	Recursos	Es posible que requiramos materiales nuevos o adicionales. Los empleados necesitarán capacitación sobre el cambio del producto.			
	Línea de tiempo	Extenderemos la línea de tiempo para tener en cuenta las fases de prueba adicionales, los ajustes en el material de marketing, etc.			
	Partes interesadas	Este cambio tendrá un impacto positivo en los clientes, pues ellos se beneficiarán de las mejoras del producto. Tendrá un impacto positivo en las partes interesadas porque mejorará la ventaja competitiva de la empresa y, posiblemente, ocasionará un incremento en los ingresos y la cuota de mercado. Tendrá un impacto mixto en los empleados porque, quizás, algunos equipos observen un aumento en su carga de trabajo.			
ANÁLISIS DE RIESGOS	Identificación de riesgos		Existen desafíos técnicos relacionados con el diseño.		Probabilidad de riesgo Bajo
	Estrategias de mitigación de riesgos	Para identificar los desafíos técnicos, realizaremos una investigación minuciosa e involucraremos a expertos antes de la implementación.			
DECISIÓN		Nombre del gerente de proyecto	Firma	Fecha	
☒	ACEPTADO				
☐	RECHAZADO	Nombre y título del responsable de la toma de decisiones	Firma	Fecha	
☐	Se solicita más información				

Formulario con campos de prioridad, versión afectada, módulo impactado, resultado de prueba y responsable.

- *Investiga sobre el Versionamiento Semántico. Coloca ejemplos reales y conocidos.*

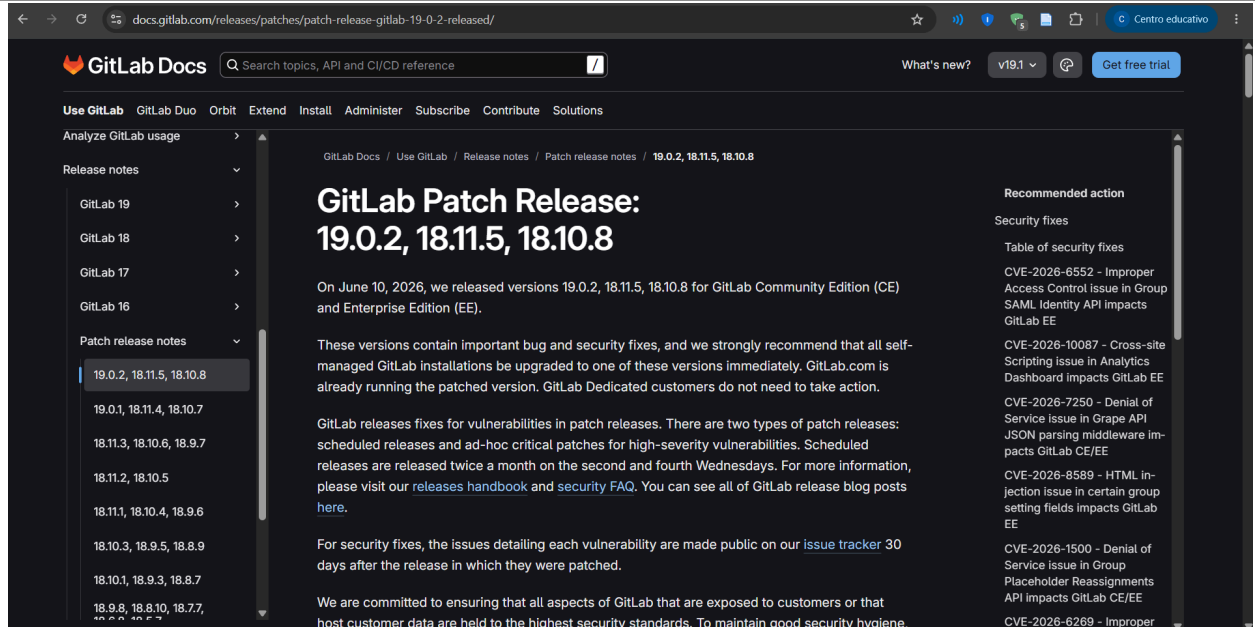
El Versionamiento Semántico o Semantic Versioning (SemVer) es un esquema de numeración de versiones basado en el formato MAJOR.MINOR.PATCH. La especificación 2.0.0 establece que el número MAJOR cambia cuando existen cambios incompatibles con versiones anteriores, MINOR cuando se agregan funcionalidades compatibles hacia atrás y PATCH cuando solo se corrigen errores.

GitLab aplica este enfoque de forma explícita. Su documentación de releases indica que sigue semantic versioning y que las versiones MAJOR pueden contener breaking changes, las MINOR nuevas funcionalidades compatibles y las PATCH únicamente bug fixes, lo que genera confianza para que muchos usuarios autoactualicen parches con menor necesidad de pruebas extensas. Esta práctica es especialmente importante en mantenimiento de software porque comunica el alcance del cambio antes incluso de instalarlo.

Ejemplos reales y conocidos

Producto o proyecto	Ejemplo de versión	Relación con SemVer
GitLab	17.5.1 / 18.8.1	Son patch releases publicadas para corregir errores o problemas de seguridad sin introducir nuevas funcionalidades mayores
React	React 19	El proyecto mantiene una política de versionado para comunicar cambios y compatibilidad entre releases estables
Composer semver	2.0.0	El cambio de versión mayor refleja una evolución con posibles incompatibilidades respecto a ramas anteriores
SemVer 2.0.0	2.0.0	La propia especificación oficial usa este formato como referencia normativa

En proyectos modernos con CI/CD, SemVer también ayuda a automatizar despliegues, dependencias y changelogs. Herramientas como Git tags, pipelines de release y generación automática de notas de versión permiten que el número de versión no sea solo un identificador, sino un mensaje técnico sobre el tipo de cambio que se está liberando.



III. CONCLUSIONES

- El estándar IEEE 1219 proporciona una estructura clara para gestionar mantenimiento de software desde la identificación del problema hasta la liberación del producto. Su aplicación permite transformar una corrección aislada en un proceso controlado, trazable y técnicamente justificable.
- GitLab constituye un sistema real adecuado para este análisis porque su documentación oficial expone de manera transparente sus políticas de releases, mantenimiento, versionado y evidencia de liberación. Esto permite vincular el estándar IEEE 1219 con prácticas reales de mantenimiento de software contemporáneo.
- El caso FINCONTROL demuestra que el mantenimiento en sistemas críticos no depende solo de corregir un error, sino de gestionar riesgo, documentación, pruebas y trazabilidad. Complementar IEEE 1219 con formularios adecuados y con versionamiento semántico mejora el control del cambio y reduce la probabilidad de impactos no deseados.

RETROALIMENTACIÓN GENERAL

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

REFERENCIAS Y BIBLIOGRAFÍA

IEEE. (1983). IEEE Std 829-1983: IEEE standard for software test documentation.

IEEE. (1987). IEEE Std 1042-1987: IEEE guide to software configuration management.

IEEE. (1998). IEEE Std 1219-1998: IEEE standard for software maintenance.

GitLab. (2026). GitLab releases. GitLab Handbook.
<https://handbook.gitlab.com/handbook/engineering/releases/>

GitLab. (s. f.). Releases. GitLab Docs. <https://docs.gitlab.com/user/project/releases/>

GitLab. (s. f.). GitLab release and maintenance policy. GitLab Docs. <https://docs.gitlab.com/policy/maintenance/>

GitLab. (s. f.). Release evidence. GitLab Docs. https://docs.gitlab.com/user/project/releases/release_evidence/

GitLab. (2024, 23 de octubre). GitLab patch release: 17.5.1 released.
<https://about.gitlab.com/releases/2024/10/23/patch-release-gitlab-17-5-1-released/>

GitLab. (2026, 19 de enero). GitLab patch release: 18.8.1.
<https://about.gitlab.com/releases/2026/01/19/gitlab-18-8-1-released/>

Preston-Werner, T. (2013). Versionado semántico 2.0.0. <https://semver.org/lang/es/>

React. (s. f.). Versioning policy. <https://react.dev/community/versioning-policy>

React Team. (2024, 5 de diciembre). React v19. <https://react.dev/blog/2024/12/05/react-19>

Composer. (s. f.). semver/CHANGELOG.md. GitHub.
<https://github.com/composer/semver/blob/main/CHANGELOG.md>